

P3800R0

2025-07 Library Evolution Poll Outcomes

Published Proposal, 2025-10-06

Authors:

[Inbal Levi - Library Evolution Chair](#) (Microsoft LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Andreas Weis - Library Evolution Assistant Chair](#) (ekxide IO GmbH)

[Billy Baker - Library Evolution Incubator Chair](#) (NVIDIA)

[Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair](#) (Argonne National Laboratory)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

3.1 Poll 1: Send "[P3125R5] constexpr pointer tagging" to Library Working Group for C++29.

3.2 Poll 2: Send "[P3778R0] Fix for type_order template definition" to Library Working Group as a DR (fix) for C++26.

References

Informative References

§ 1. Introduction

In 2025-07, the C++ Library Evolution group conducted a series of electronic decision polls [\[P3799R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 15 people participated in the polls. Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send " [P3125R5] constexpr pointer tagging" to Library Working Group for C++29.	10	2	1	0	1	Consensus in favor
Poll 2: Send " [P3778R0] Fix for type_order template definition" to Library Working Group as a DR (fix) for C++26.	13	1	1	0	0	Consensus in favor

All the polls have consensus in favor and the papers will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[\[P3125R5\]](#) constexpr pointer tagging" to Library Working Group for C++29.

This helps prevent users from invoking UB by breaking pointer provenance rules when doing widely used pointer tagging techniques.

— Strongly Favor

A useful facility to legitimize a very common programming technique.

— Strongly Favor

This feature covers a niche for which user code exists but is probably bug-prone in many cases. My only worry was how the API managed the case of user code requesting more bits than can be provided and I can see from the wording part that this is handled reasonably.

— Strongly Favor

Editorial Comments: The declaration (in 4 places):

```
static constexpr bits_available = see below;
```

Is missing a type. There should be a `unsigned` after the `constexpr` (and maybe also an `inline`, too?). Also, the *Returns:* clause for `from_tagged` says "an object `dp` of type `pointer_tag_type`" instead of "type `pointer_tag_pair`". The return type need not be repeated in the text at all; it would be sufficient to say "An object `dp` such that:"

— Strongly Favor

This feature covers a niche for which user code exists but is probably bug-prone in many cases. My only worry was how the API managed the case of user code requesting more bits than can be provided and I can see from the wording part that this is handled reasonably.

— Strongly Favor

It's standardizing existing practice.

— Weakly Favor

It is already used and have reasonable use cases. Standardizing it seems to be beneficial.

— Weakly Favor

It's unclear to me how this facility makes `constexpr atomic` > work. Lacking that understanding, the motivation for standardizing this facility is unclear to me.

— Neutral

While I do think the proposal is well motivated and pointer tagging is extremely useful, this iteration of the design could be improved without too much effort to support forward declarations, opaque ptrs, nested tagged pointers and other pointer-like types, by adding a trait to extract/construct a tagged pointer from user-defined types. Such a trait would only need the following additional members `template struct pointer_tag_traits { static inline void *get_as_void(T *P) { return P; } static inline T * get_from_void(void *P) { return static_cast (P); } static constexpr int bits_available = log2(alignof(T)); };`

Furthermore, that the trait is specialized for T rather than T* makes any improvement impossible and it is the wrong mental model (a tagged pointer is an optimization of pair, not a pointer to T). Finally, we should specify that the tag uses the high-most available low bits (so that any unused remaining low bits are still accessible for composition and nesting. Failing to make this type composable/customisable will prevent adoption in existing code base, as everything described therein is standard practice.

— Strongly Against

Editor's Note: The feature is targeting C++29, the commenter proposed to bring a follow-up paper for the design changes requested.

§ 3.2. Poll 2: Send "[P3778R0] Fix for type_order template definition" to Library Working Group as a DR (fix) for C++26.

Seems like a reasonable approach to a real and pressing problem. Thanks!

— Strongly Favor

We can fix `strong_ordering` later. This paper a strict improvement over the status quo.

— Strongly Favor

This is an obviously correct fix to make a badly needed feature work.

— Strongly Favor

Bugfix to make the feature implementable. The proposed resolution seems reasonable.

— Weakly Favor

Implementations seem to be able to handle it, at least Clang. It seems unnecessary.

— Neutral

§ References

§ Informative References

[P3125R5]

Hana Dusíková. *constexpr pointer tagging*. 06 October 2025. URL: <https://wg21.link/P3125r5>

[P3778R0]

Gašper Ažman. *Fixing `type\_order` template definition*. 3 July 2025. URL: <https://wg21.link/p3778r0>

[P3799R0]

Inbal Levi; et al. *2025-07 Library Evolution Polls*. 15 July 2025. URL: <https://wg21.link/P3799r0>